DATA STRUCTURES AND ALGORITHMS I

A concise, concept-first study guide

Course: CSE 2215 | Basis: Uploaded UIU course syllabus

Core definitions • essential formulas • operation logic • complexity tables • exam-ready pseudocode

How to use this guide: First understand the idea and the invariant; then trace each algorithm by hand; finally implement it in code and explain its time and space complexity.

Prepared as an original study aid. The organization and explanations are independently written, with CLRS (Cormen, Leiserson, Rivest & Stein) used as a conceptual reference alongside the syllabus-listed resources.

How the course fits together

A data structure organizes information so that operations such as access, search, insertion, deletion, and traversal can be performed efficiently. An algorithm is a finite, precise sequence of steps that solves a problem.

Course block	Main question you should be able to answer
Analysis and recursion	How does resource use grow as input size grows? How can a problem be reduced to smaller instances?
Arrays, searching, sorting	How is data stored contiguously, and how can it be searched or ordered?
Linked lists, stacks, queues	How do pointer-based sequences and restricted-access structures work?
Trees, BSTs, heaps	How can hierarchical data support traversal, ordered search, or priority selection?
Graphs	How can relationships be represented and explored?
Disjoint sets	How can we track which items belong to the same connected group?

Learning outcomes

- Choose an appropriate data structure for a problem and justify the trade-off.
- Trace and implement searching, insertion, deletion, sorting, and traversal operations.
- Analyze best-, average-, and worst-case running time using asymptotic notation.
- Work confidently with linear structures (arrays, lists, stacks, queues) and non-linear structures (trees and graphs).

Reference approach

The syllabus names Introduction to Algorithms by Cormen, Leiserson, Rivest, and Stein (CLRS) and other data-structures texts. This guide uses standard terminology and original explanations; it is a summary, not a substitute for full textbook proofs or instructor-specific conventions.

Notation: n = number of elements; V = number of vertices; E = number of edges; h = tree height. Unless stated otherwise, complexity refers to asymptotic growth as input size becomes large.

1. Complexity analysis

What complexity measures

Time complexity estimates how the number of elementary operations grows with input size. Space complexity estimates additional memory used. We focus on growth rates rather than machine-specific seconds.

Notation	Meaning	Interpretation
$O(g(n))$	Asymptotic upper bound	The function grows no faster than a constant multiple of g(n), eventually.
$\Omega(g(n))$	Asymptotic lower bound	The function grows at least as fast as a constant multiple of g(n), eventually.
$\Theta(g(n))$	Tight asymptotic bound	Both an $O(g(n))$ upper bound and $\Omega(g(n))$ lower bound hold.
$o(g(n))$, $\omega(g(n))$	Strict upper/lower bounds	Useful for formal comparisons; usually less central in introductory tracing.

Rules of thumb

- Sequential statements: add costs; the dominant term determines the asymptotic class.
- Nested loops: multiply iteration counts when each loop runs independently across its full range.
- Halving/doubling patterns often produce logarithmic counts: $\log_2 n$.
- Drop constants and lower-order terms: $3n^2 + 8n + 4 = \Theta(n^2)$.
- Best case is the most favorable input of size n; worst case is the least favorable; average case depends on an assumed input distribution.

Common growth rates

Growth	Typical example	Relative scalability
$O(1)$	Access array element by index	Constant
$O(\log n)$	Binary search in a sorted array	Very good
$O(n)$	Linear search; one full traversal	Good
$O(n \log n)$	Merge sort; heap sort	Good for large inputs
$O(n^2)$	Selection sort; worst-case insertion sort	Poor for large n
$O(2^n)$	Naive recursive subset-like search	Very expensive
$O(n!)$	Try every permutation	Extremely expensive

Loop examples

```
for i = 1 to n:
    do_constant_work()       // Θ(n)

for i = 1 to n:
    for j = 1 to n:
        do_constant_work()  // Θ(n²)

i = 1
while i < n:
    i = 2 * i                // Θ(log n)
```

Always explain the reason, not just the label: for example, the last loop doubles i each time, so after k iterations i ≈ 2^k; reaching n requires k ≈ $\log_2 n$.

2. Recursion

A recursive algorithm calls itself on a smaller instance of the same problem. Every correct recursive design needs a base case and progress toward that base case.

Design checklist

- Base case: the smallest input that can be answered directly.
- Recursive case: reduce the problem and combine the returned result.
- Progress: each call must move closer to a base case.
- Trace the call stack: calls pause until their recursive calls return.

Factorial example

```
factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)
```

For $n \geq 1$, $n! = n \times (n-1) \times \ldots \times 2 \times 1$. The recurrence is $T(n) = T(n-1) + \Theta(1)$, so time is $\Theta(n)$ and call-stack space is $\Theta(n)$.

Tower of Hanoi

Move n disks from source to destination using an auxiliary peg, moving one disk at a time and never placing a larger disk on a smaller one.

```
Hanoi(n, source, auxiliary, destination):
    if n == 1:
        move source -> destination
        return
    Hanoi(n-1, source, destination, auxiliary)
    move source -> destination
    Hanoi(n-1, auxiliary, source, destination)
```

Minimum moves: $T(n) = 2T(n-1) + 1$, $T(1) = 1$, hence $T(n) = 2^n - 1$. The growth is $\Theta(2^n)$.

Recursion versus iteration

Recursion often makes divide-and-conquer or self-similar logic clearer, but uses call-stack memory and may incur function-call overhead. Iteration can reduce stack use. Choose the version that is clear, safe, and appropriate for the constraints.

3. Arrays, vectors, and searching

Arrays and memory mapping

An array stores same-type elements in contiguous memory. For a zero-based array with element size w and base address B, the address of A[i] is:

Address(A[i]) = B + i × w (valid for 0 ≤ i < n)

- Index access/update: Θ(1).
- Search in an unsorted array: Θ(n) worst case.
- Insertion/deletion in the middle generally shifts elements: Θ(n).

Vector / dynamic array

A vector is a resizable array. When capacity is exhausted, it allocates a larger block and copies elements. Appending is Θ(1) amortized, although an individual resize can cost Θ(n). Random access remains Θ(1); insertion near the beginning remains Θ(n).

Linear search

```
linearSearch(A, n, key):
    for i = 0 to n-1:
        if A[i] == key:
            return i
    return NOT_FOUND
```

Best case Θ(1) (first element matches); worst case Θ(n) (last or absent); average case Θ(n) under common assumptions.

Binary search

Requires sorted data and efficient access to the middle element. Each comparison discards about half of the remaining search range.

```
binarySearch(A, n, key):
    low = 0; high = n - 1
    while low <= high:
        mid = low + floor((high - low) / 2)
        if A[mid] == key: return mid
        if A[mid] < key: low = mid + 1
        else: high = mid - 1
    return NOT_FOUND
```

Time: best Θ(1), worst Θ(log n). Iterative extra space: Θ(1). Avoid (low + high) / 2 in languages where integer overflow is possible; the expression above is safer.

Common exam trap: binary search is not generally appropriate for a linked list, because reaching the middle node is not Θ(1).

4. Sorting algorithms

Sorting rearranges elements according to a key. Stability means equal-key items keep their original relative order. In-place means the algorithm uses only a small amount of extra storage, ignoring the input array.

Algorithm	Best time	Average time	Worst time	Extra space	Stable? / key idea
Insertion sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$	Yes if implemented carefully; insert each item into sorted prefix.
Selection sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(1)$	Usually no; repeatedly select the minimum.
Counting sort	$\Theta(n + k)$	$\Theta(n + k)$	$\Theta(n + k)$	$\Theta(n + k)$	Can be stable; k is key-range size.

Insertion sort

```
for i = 1 to n-1:
    key = A[i]
    j = i - 1
    while j >= 0 and A[j] > key:
        A[j+1] = A[j]
        j = j - 1
    A[j+1] = key
```

Maintains the invariant that $A[0..i-1]$ is sorted before each iteration. Excellent for small or nearly sorted arrays; worst-case shifts make it quadratic.

Selection sort

```
for i = 0 to n-2:
    minIndex = i
    for j = i+1 to n-1:
        if A[j] < A[minIndex]:
            minIndex = j
    swap(A[i], A[minIndex])
```

Makes $\Theta(n^2)$ comparisons regardless of input order, but only $O(n)$ swaps. It is simple, but usually slower than advanced sorting for large arrays.

Counting sort — linear-time sorting

For integer keys in a known, manageable range $0…k$, count occurrences and use cumulative counts to place records. It is not comparison-based, so it can beat the $\Omega(n \log n)$ comparison-sorting lower bound when k is suitably small.

- Time and space: $\Theta(n + k)$.
- If k is much larger than n, it may be wasteful.
- To make the standard output-placement version stable, process input from right to left when using cumulative counts.

Comparison-sorting lower bound: any general comparison sort requires $\Omega(n \log n)$ comparisons in the worst case. This bound does not apply to non-comparison methods under their key assumptions.

5. Linked lists

A linked list stores nodes connected by pointers; nodes need not be adjacent in memory. Each node stores data and one or more links.

Structure	Node links	Strength	Limitation
Singly linked	data, next	Low overhead; insert/delete at head in Θ(1)	No direct backward traversal; search/access by position Θ(n)
Doubly linked	prev, data, next	Traverse both directions; delete a known node in Θ(1)	More memory and pointer updates
Circular linked	last links back to first	Useful for cyclic iteration and round-robin tasks	Traversal must stop after returning to start

Core operations

- Creation: allocate nodes and connect them in the required order.
- Display/traversal: begin at head and follow next until NULL (or until returning to the start in a circular list).
- Insertion at head: new.next = head; head = new.
- Insertion after node p: new.next = p.next; p.next = new.
- Deletion at head: head = head.next, then release the old node.
- Deletion by value in a singly list: find the target and its predecessor, reconnect predecessor.next to target.next, then release target.

```
search(head, key):
    current = head
    while current != NULL:
        if current.data == key: return current
        current = current.next
    return NULL
```

Complexity snapshot

Access/search by position: Θ(n). Insert/delete at head: Θ(1). Insert after a known node: Θ(1). Finding the position or predecessor first generally costs Θ(n).

Pointer safety: handle an empty list, a one-node list, head changes, tail changes, and memory release. Many linked-list bugs are boundary-case bugs.

6. Stacks and queues

Stack — LIFO

Last In, First Out. The most recently pushed item is the first popped. Main operations: push, pop, peek/top, isEmpty.

Operation	Array implementation	Linked-list implementation
push / enqueue-like insertion	Θ(1) amortized for dynamic array; Θ(1) if capacity available	Θ(1) at head
pop	Θ(1)	Θ(1) at head
peek/top	Θ(1)	Θ(1) at head
Space	Θ(n)	Θ(n), plus pointer overhead

Array stack model

```
top = -1

push(x):
    if top == capacity - 1: report OVERFLOW
    else:
        top = top + 1
        A[top] = x

pop():
    if top == -1: report UNDERFLOW
    else:
        x = A[top]
        top = top - 1
        return x
```

Queue — FIFO

First In, First Out. Enqueue adds at the rear; dequeue removes from the front. Applications include scheduling, buffering, breadth-first search, and request processing.

- A simple array queue can waste space after repeated dequeues; a circular array reuses slots.
- Circular queue (capacity m): next index = (index + 1) mod m.
- In a linked queue, keep front and rear pointers. Enqueue at rear and dequeue at front in Θ(1).

```
enqueue(x):
    if queue is full: report OVERFLOW
    rear = (rear + 1) mod capacity
    A[rear] = x

dequeue():
    if queue is empty: report UNDERFLOW
    x = A[front]
    front = (front + 1) mod capacity
    return x
```

The exact full/empty test for a circular queue depends on the chosen convention (for example, storing a count or leaving one slot unused).

Stack applications

- Function calls and recursion: activation records are managed using a call stack.
- Balanced delimiters and syntax parsing.
- Infix-to-postfix conversion and postfix evaluation.
- Undo operations, depth-first search, and backtracking.

7. Expressions: infix, postfix, and evaluation

Infix places operators between operands (A + B). Postfix places operators after operands (A B +), so parentheses are not needed when operator arity is known.

Infix to postfix: operator-stack rules

- Send operands directly to output.
- Push '('. On ')' pop operators to output until '(' is removed.
- For an operator, pop higher-precedence operators; also pop equal-precedence operators when the incoming operator is left-associative. Then push the incoming operator.
- At the end, pop remaining operators. Precedence and associativity must be defined, including the special right-associativity of exponentiation if used.

Typical precedence

Level	Operators	Associativity
High	Parentheses control grouping	—
1	^ (if included)	Often right-associative
2	*, /, %	Usually left-associative
3	+, −	Usually left-associative

Postfix evaluation

```
for each token:
    if token is an operand:
        push(token)
    else:
        right = pop()
        left  = pop()
        push(left operator right)
answer = pop()
```

For subtraction and division, operand order matters: pop the right operand first, then the left. A valid expression should leave exactly one final value on the stack.

Example

Infix: (3 + 4) × 2. Postfix: 3 4 + 2 ×. Evaluation: push 3, push 4, compute 3 + 4 = 7, push 2, compute 7 × 2 = 14.

8. Trees and binary trees

A tree is a connected, acyclic hierarchical structure. A node may have a parent and zero or more children. The root has no parent; leaves have no children.

Terminology and properties

- Depth: number of edges from the root to a node. Height of a node: longest downward path in edges to a leaf.
- A binary tree has at most two children per node, conventionally called left and right.
- A full/proper binary tree has either zero or two children per node. A complete binary tree fills levels left to right, with only the last level possibly incomplete.
- For a binary tree with n nodes, there are $n - 1$ edges (for $n \geq 1$).
- At level d, a binary tree can contain at most 2^d nodes (root at level 0).
- A binary tree of height h can contain at most $2^{(h+1)} - 1$ nodes.

Representations

Array representation works especially well for complete trees. With zero-based indexing: $left(i) = 2i + 1$, $right(i) = 2i + 2$, $parent(i) = floor((i - 1)/2)$ for $i > 0$. Pointer representation stores data and left/right child references and suits irregular trees.

Tree traversals

Traversal	Order	Use / pattern
Preorder	Root, Left, Right	Copy/serialize tree; prefix-like output
Inorder	Left, Root, Right	Outputs keys in sorted order for a BST
Postorder	Left, Right, Root	Delete/free a tree; evaluate expression tree
Level order	Breadth-first by level	Uses a queue

```
preorder(node):
    if node == NULL: return
    visit(node)
    preorder(node.left)
    preorder(node.right)

inorder(node):
    if node == NULL: return
    inorder(node.left)
    visit(node)
    inorder(node.right)

postorder(node):
    if node == NULL: return
    postorder(node.left)
    postorder(node.right)
    visit(node)
```

Each traversal visits each node once: $\Theta(n)$ time. Recursive extra space is $\Theta(h)$ for the call stack; in the worst skewed tree, $h = \Theta(n)$.

9. Binary Search Trees (BSTs)

A BST is a binary tree with an ordering rule: keys in the left subtree are less than the node key, and keys in the right subtree are greater, assuming duplicates are handled by a stated policy.

Operations

- Search: compare the target with the current node; continue left or right.
- Insertion: follow the search path until a NULL child position is found.
- Minimum/maximum: follow left/right links until no further child exists.
- Deletion has three cases:
- 1. Leaf: remove the node.
- 2. One child: connect the parent directly to the node's child.
- 3. Two children: replace the node's key with its inorder successor (minimum of right subtree) or predecessor (maximum of left subtree), then delete that replacement node.

Complexity depends on height

Operation	Balanced-ish tree	Worst-case skewed tree
Search	O(log n)	O(n)
Insert	O(log n)	O(n)
Delete	O(log n)	O(n)
Traversal	Θ(n)	Θ(n)

A BST is not automatically balanced. Inserting sorted keys into an ordinary BST can produce a chain, so operations may degrade to linear time.

Applications

Ordered lookup, maintaining sorted data, predecessor/successor queries, and range queries. Self-balancing BSTs improve worst-case height, but their balancing rules are outside the core syllabus summary unless taught separately.

10. Heaps, heap sort, and priority queues

A binary heap is a complete binary tree stored efficiently in an array. In a max-heap, every parent key is at least as large as its children; in a min-heap, every parent is at most its children.

Index relationships (zero-based)

left(i) = 2i + 1 right(i) = 2i + 2 parent(i) = floor((i − 1)/2), for i > 0

Operations and costs

Operation	Typical time	Why
Peek max/min	Θ(1)	Root holds the extreme key
Insert	O(log n)	Add at end, sift up
Extract max/min	O(log n)	Move last item to root, sift down
Build heap bottom-up	Θ(n)	Most nodes are near the leaves
Heap sort	Θ(n log n)	Repeatedly extract the extreme

Heap sort (max-heap for ascending order)

- Build a max-heap from the array.
- Swap the root (maximum) with the last element in the heap.
- Shrink the heap by one and restore the heap property with sift-down.
- Repeat until one element remains.

Heap sort is in-place in its standard array form and has Θ(n log n) best, average, and worst-case time. Standard heap sort is not stable.

Priority queue

A priority queue removes the item with highest (or lowest) priority, rather than the earliest inserted item. A heap is a common implementation. Applications include CPU/task scheduling, event simulation, graph algorithms, and selecting the next most urgent item.

11. Graphs, representation, BFS, and DFS

A graph G = (V, E) contains vertices and edges. Graphs may be directed or undirected, weighted or unweighted. A path is a sequence of connected vertices; a cycle is a path that returns to its starting vertex.

Representation

Representation	Space	Best fit / trade-off
Adjacency matrix	$\Theta(V^2)$	Fast edge-existence check $\Theta(1)$; wastes space for sparse graphs.
Adjacency list	$\Theta(V + E)$	Space-efficient for sparse graphs; iterate a vertex's neighbors efficiently.

For an undirected graph, each edge normally appears in two adjacency lists. For a directed graph, store outgoing neighbors. Self-loops and parallel edges depend on the graph model.

Breadth-first search (BFS)

BFS explores vertices in layers from a source using a queue. In an unweighted graph, it finds shortest paths measured by number of edges.

```
BFS(G, source):
    mark all vertices unvisited
    mark source visited; enqueue(source)
    while queue is not empty:
        u = dequeue()
        for each v adjacent to u:
            if v is unvisited:
                mark v visited
                parent[v] = u
                enqueue(v)
```

Depth-first search (DFS)

DFS follows one path as far as possible before backtracking. It can be implemented recursively or with an explicit stack. It supports connected-component discovery, cycle detection, and topological sorting.

```
DFS-Visit(u):
    mark u visited
    for each v adjacent to u:
        if v is unvisited:
            parent[v] = u
            DFS-Visit(v)
```

Representation	BFS time	DFS time
Adjacency list	$O(V + E)$	$O(V + E)$
Adjacency matrix	$O(V^2)$	$O(V^2)$

Mark a vertex visited when it is enqueued in BFS, not only when it is dequeued; this prevents duplicate queue entries in the standard implementation.

12. Graph applications, DAGs, and topological sort

Common applications

- Computer and communication networks; road maps and route planning.
- Dependency scheduling (courses, tasks, build systems).
- Social networks, web links, and recommendation relationships.
- State-transition models and resource-allocation relationships.

Directed acyclic graph (DAG)

A DAG is a directed graph with no directed cycle. Its edges can represent prerequisite or dependency constraints.

Topological ordering

A topological order is a linear ordering of vertices such that for every directed edge u → v, u appears before v. It exists if and only if the directed graph is acyclic.

Kahn's algorithm (indegree method)

```
compute indegree[v] for every vertex v
put every vertex with indegree 0 into a queue
order = empty list

while queue is not empty:
    u = dequeue()
    append u to order
    for each edge u -> v:
        indegree[v] = indegree[v] - 1
        if indegree[v] == 0:
            enqueue(v)

if length(order) < V:
    report "cycle exists; no topological order"
else:
    output order
```

With adjacency lists, time is O(V + E). Multiple valid orders may exist. A graph containing a cycle cannot have a topological ordering.

Important distinction: BFS/DFS are traversals; topological sorting is an ordering subject to directed-edge constraints. Kahn's algorithm uses a queue and indegrees.

13. Search algorithms and choosing structures

The syllabus includes search algorithms; the key introductory methods here are linear search and binary search, with graph search handled by BFS and DFS.

Problem / data	Suitable approach	Typical cost
Unsorted array	Linear search	O(n)
Sorted array with random access	Binary search	O(log n)
Ordered lookup in a BST	Follow comparison path	O(h); h may be n
Reach vertices by distance layers	BFS	O(V + E) with adjacency lists
Explore paths / components	DFS	O(V + E) with adjacency lists
Repeatedly obtain min/max priority	Heap-based priority queue	Peek O(1), insert/extract O(log n)

A practical decision method

- Identify the operations the problem performs most often.
- Check the data assumptions: sorted or unsorted, dense or sparse graph, bounded integer keys, ordered versus priority access.
- Compare time, extra space, and implementation complexity.
- State the trade-off explicitly; no single structure is best for every operation.

14. Set operations: Make-Set, Find, Union

A disjoint-set data structure (also called Union-Find) maintains a collection of non-overlapping sets. Each element belongs to exactly one set.

Core operations

- Make-Set(x): create a new set containing only x.
- Find(x): return the representative of the set containing x.
- Union(x, y): merge the two sets containing x and y, if they are different.

Forest representation

Each set is represented as a rooted tree; parent pointers lead toward a representative root. Initially, each element is its own root.

```
MAKE-SET(x):
    parent[x] = x
    rank[x] = 0

FIND(x):
    if parent[x] != x:
        parent[x] = FIND(parent[x])   // path compression
    return parent[x]

UNION(x, y):
    rx = FIND(x)
    ry = FIND(y)
    if rx == ry: return
    attach the lower-rank root under the higher-rank root
    if ranks are equal:
        choose one root as parent
        increase its rank by 1
```

Path compression flattens paths during Find. Union by rank/size avoids attaching a large/deep tree beneath a smaller one. Together, these optimizations make a sequence of operations nearly constant amortized time; the formal bound uses the inverse Ackermann function $\alpha(n)$.

Applications

- Tracking connected components in an undirected graph.
- Kruskal's minimum spanning tree algorithm.
- Merging groups and checking whether two elements are connected.

15. Complexity quick-reference

Structure / algorithm	Search / access	Insert	Delete / remove	Notes
Array (index)	Access $\Theta(1)$; search $O(n)$	End: $O(1)$ amortized for vector; middle $O(n)$	$O(n)$ if shifting	Contiguous storage
Singly linked list	$O(n)$	Head $O(1)$; after known node $O(1)$	Head $O(1)$; by value $O(n)$	Pointer overhead
Stack	Top $O(1)$	Push $O(1)$	Pop $O(1)$	LIFO
Queue	Front $O(1)$	Enqueue $O(1)$	Dequeue $O(1)$	FIFO
BST	$O(h)$	$O(h)$	$O(h)$	Balanced $h \approx \log n$; skewed $h \approx n$
Binary heap	Find arbitrary key $O(n)$	$O(\log n)$	Extract extreme $O(\log n)$	Peek extreme $O(1)$
Adjacency matrix	Edge test $O(1)$	Edge add $O(1)$	Edge remove $O(1)$	Space $\Theta(V^2)$
Adjacency list	Neighbor scan $O(\deg(v))$	Add edge typically $O(1)$	Depends on representation	Space $\Theta(V+E)$

Algorithm complexity recap

Algorithm	Best	Average	Worst
Linear search	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Binary search	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$
Insertion sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$
Selection sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$
Counting sort	$\Theta(n+k)$	$\Theta(n+k)$	$\Theta(n+k)$
BFS / DFS adjacency list	—	—	$O(V+E)$
Heap sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$
Topological sort (Kahn)	—	—	$O(V+E)$

16. What you should be able to do after the course

Use this as a self-checklist. You are ready to move on when you can explain the idea in your own words, trace a small example, implement the operation, and justify its complexity.

- ☐ Explain Big-O, Big-Ω, and Big-Θ; analyze single loops, nested loops, and logarithmic loops.
- ☐ Identify best, average, and worst cases; state time and auxiliary-space complexity.
- ☐ Write and trace a recursive function; solve the Tower of Hanoi recurrence and know $2^n - 1$ moves.
- ☐ Calculate array addresses and explain dynamic-array resizing and amortized append.
- ☐ Implement linear search, binary search, insertion sort, selection sort, and counting sort.
- ☐ Create, traverse, search, insert into, and delete from singly, doubly, and circular linked lists.
- ☐ Implement stack and queue using arrays and linked lists; handle overflow, underflow, and circular indexing.
- ☐ Convert infix expressions to postfix and evaluate postfix expressions with a stack.
- ☐ Represent trees with arrays or pointers; trace preorder, inorder, postorder, and level-order traversal.
- ☐ Implement BST search, insertion, and all three deletion cases; explain how height affects complexity.
- ☐ Explain heap properties, array index formulas, heap insertion/extraction, heap sort, and priority queues.
- ☐ Represent graphs using matrices and lists; implement BFS and DFS and compare their costs.
- ☐ Explain DAGs and run Kahn's topological-sort algorithm, including cycle detection.
- ☐ Implement Make-Set, Find, and Union and explain path compression and union by rank/size.

Suggested study sequence

1. Complexity and recursion → 2. Arrays, searching, sorting → 3. Linked lists → 4. Stacks and queues → 5. Expression processing → 6. Trees and BSTs → 7. Heaps → 8. Graphs, BFS/DFS, DAGs → 9. Disjoint sets.

Recommended reference

Cormen, Leiserson, Rivest, and Stein, Introduction to Algorithms (CLRS), 4th edition. Use it for deeper analysis and formal treatment; pair it with lecture notes and implementation practice. The topic coverage and sequence in this PDF follow the uploaded CSE 2215 syllabus.

Final advice: for every topic, practise three things: (1) draw the structure, (2) trace the algorithm on a small input, and (3) explain why the time complexity has that order of growth.